



## Why Jenkins?

### *Introduction*

Jenkins is one of the most popular **Continuous Integration (CI)** and **Continuous Deployment (CD)** tools.

- Why do software teams use Jenkins?
- What problems does Jenkins solve in DevOps?
- How does Jenkins improve automation and efficiency?

This guide explains the **importance of Jenkins in modern software development**.

---

### *Section 1: Learn*

#### What Problem Does Jenkins Solve?

Before Jenkins, developers **manually tested and deployed code**, leading to **delays, errors, and inconsistencies**.

| Without Jenkins                   | With Jenkins                          |
|-----------------------------------|---------------------------------------|
| Manual testing and deployment     | Automated CI/CD pipelines             |
| Human errors in software releases | Automated error detection             |
| Slow software development cycles  | Fast and frequent releases            |
| Hard to integrate different tools | Works with Git, Docker, AWS, and more |



## Why Do Companies Use Jenkins?

- **Automation** – Reduces human intervention in testing and deployment.
- **Early Bug Detection** – Runs tests automatically, identifying issues early.
- **Faster Releases** – Supports continuous software delivery with CI/CD.
- **Scalability** – Works with multiple machines for parallel builds.
- **Extensive Plugin Support** – Over **1,500+ plugins** for various integrations.

## Real-Life Example

A **banking company** struggled with **slow and error-prone manual deployments**. After switching to **Jenkins**, they **automated their workflows**, reducing deployment time **from 8 hours to 30 minutes**, saving both **time and costs**.

---

## Section 2: Practice

### 1. Checking If Jenkins Is Installed

To verify if Jenkins is installed, run:

```
jenkins --version
```

To check via the UI:

1. Open **Jenkins Dashboard** → Click **Manage Jenkins**.
  2. Scroll down to **System Information** → Find **Jenkins Version**.
-



## 2. Running a Simple Jenkins Job

### Step 1: Create a New Job

1. Click **"New Item"** on the Jenkins dashboard.
2. Enter a **Job Name** → Select **Freestyle Project** → Click **OK**.

### Step 2: Configure the Job

1. **Build Steps** – Click **"Add build step"** → **"Execute shell"** → Enter:

```
echo "Hello, Jenkins!"
```

2. Click **Save**.

### Step 3: Run the Job

1. Click **"Build Now"**.
2. Open **"Console Output"** to check the execution log.

Why is this useful?

- Confirms that Jenkins is correctly installed and running.

---

## 3. Setting Up an Automated CI/CD Pipeline

1. Click **"New Item"** → Select **Pipeline** → Click **OK**.
2. In the Pipeline section, enter:

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        echo 'Building application...'      }  
    }  
  }  
}
```



```
    }  
  }  
  stage('Test') {  
    steps {  
      echo 'Running tests...'  
    }  
  }  
  stage('Deploy') {  
    steps {  
      echo 'Deploying application...'  
    }  
  }  
}  
}
```

2. Click Save → Build Now.

Why is this important?

- Automates software testing and deployment in real-time.

---

### Section 3: Know More

#### Frequently Asked Questions

1. What is Jenkins used for?
  - Automates build, test, and deployment workflows in software development.
2. How does Jenkins help in DevOps?



- Jenkins enables Continuous Integration (CI) and Continuous Deployment (CD), allowing faster and error-free software releases.
3. Why is Jenkins better than manual deployment?
    - Reduces human errors, speeds up releases, and integrates easily with DevOps tools.
  4. What are some alternatives to Jenkins?
    - GitHub Actions, GitLab CI, CircleCI, and Travis CI.
  5. Where can I learn Jenkins?
    - Official Jenkins Documentation: [jenkins.io](https://jenkins.io)
- 

## ***Conclusion***

Jenkins revolutionizes software automation by integrating CI/CD workflows. Start by installing Jenkins, running jobs, and automating deployments, and soon you'll be optimizing software releases like a pro!